

Chapter 6 Synchronization Tools

Outline

- Background
- The critical-section Problem
- Peterson's Solution
- Hardware Support for Synchronization
- Mutex Locks
- Semaphores
- Monitors
- Liveness
- Evaluation

Objectives

- Describe the critical-section problem and illustrate a race condition
- Illustrate hardware solutions to the critical-section problem using memory barriers, compare-and-swap operations and atomic variables
- Demonstrate how mutex locks, semaphores, monitors and condition variables can be used to solve the critical section problem
- Evaluation tools that solve the critical-section problem in low-, moderate- and high-contention scenarios.

Background

- Processes can execute **concurrently**
- May be interrupted at any time, partially completing execution
- Concurrent access to shared data may result in **data inconsistency**. Thus require mechanisms to ensure the orderly execution of cooperating processes

Race Condition

If two processes create child process at the same time, race condition on kernel variable `next_available_pid` will occurs. That is, **the same pid could be assigned to two different processes**.

Critical Section Problem

Consider the system of n processes and each process has **critical section** segment of code. That is, process may be changing common variables, updating table, etc. When one process is in critical section, no other may be in its critical section.

Critical section problem is to design protocol to solve this problem.

For each process,

entry section $\rightarrow$ (asked permission) critical section $\rightarrow$ exit section $\rightarrow$ remainder section

General structure of process:

```
while (true) {
```

entry section

critical section

exit section

remainder section

```
}
```

Requirements for solution to critical-section problem

- mutual exclusion for critical section
- progress: if no process in the critical section and there exist some processes wishing to enter their critical section, the selection of process **cannot be postponed infinitely**.
- bounded waiting: After a process requires to enter the critical section and before the request is permitted, a bound must exist on the number of times that other processes are allowed to enter their critical sections.

Interrupt-based Solution

Entry section: disable interrupts

Exit section: enable interrupts

But still some problems

Software Solution 1

The solution is for two processes. Assume load and store are atomic(i.e., cannot be interrupted). The two processes share one variable and the variable indicates whose turn it is to enter the critical section.

For P_i , the algorithm:

```
while (true) {
    while (turn == j);
    /* critical section */
    turn = j;
    /* remainder section */
}
```

The solution satisfies mutual exclusion but not for progress and bounded waiting.

Peterson's Solution

The solution is for two processes and also assume load and store are atomic.

The two processes share two variables, one `turn` and one `flag[2]`. `turn` indicates whose turn it is enter the critical section. `flag` array is used to indicate if a process is ready to enter the critical section.

For P_i , the algorithm:

```
while (true) {
    flag[i] = true;
    turn = j;
    while (flag[j] && turn == j);
    /*critical section*/
    flag[i] = false;
    /*remainder section*/
}
```

The solution satisfied mutual exclusion, progress and bounded waiting.

However, it is not guaranteed to work on modern architecture. For single-threaded this is ok but for multithreaded the **reordering** may produce inconsistent or unexpected results.

So to ensure Peterson's solution will work correctly on modern computer we must use **memory barrier**.

Memory Barrier

Memory model are the memory guarantees a computer architecture makes to application programs.

- Strongly ordered: a memory modification of one processor is **immediately** visible to all other processors
- Weakly ordered: a memory modification of one processor **may not be immediately** visible to all other processors.

A **memory barrier** is an instruction that forces any change in memory before the barrier to be propagated(made visible) to all other processors.

Memory Barrier Instructions

When a memory barrier instruction is performed, the system **ensures that all loads and stores are completed** before any subsequent load or store operations are performed.

That ensures the store operations are completed in memory and visible to other processors before future load or store operations are performed even the instructions is reordered.

Synchronization Hardware

Many systems provide hardware support for implementing the critical section code. For example, uniprocessors could disable interrupts. However, it is generally too inefficient on multiprocessor systems. and OSs using this not broadly scalable.

Hardware Instructions

- Test-and-Set instruction
Test and modify the content of a word
- Compare-and-Swap instruction
Swap the contents of two words atomically

Test-and-Set instruction

- Executed atomically
- Returns the original value of passed parameter
- Set the new value of passed parameter to `true` .

```
boolean test_and_set (boolean *target){
    boolean rv = *target;
    *target = true;
    return rv;
}
```

Compare-and-swap instruction